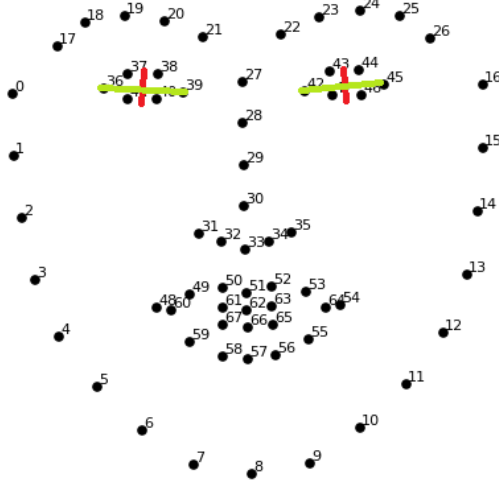


## Göz Kırpma Algılama

Göz bölgesi üzerinde altışar nokta (sol 36,37,38,39,40,41 , sağ 42,43,44,45,46,47) bulunmaktadır. En sol (sol 36 sağ 42) ve en sağ (sol 39, 45) noktalar arasındaki uzaklık (yeşil) ile üstteki (sol 37,38 sağ 43,44) noktaların ortası ve alttaki (sol 40,41 sağ 46 47) noktaların ortasının arasındaki uzaklığı (kırmızı) yatay/dikey şeklinde oranlıyoruz. Bu çıkan orana göre gözün kapalı olup olmadığı bulunur.



```
def midPoint(p1, p2):
    # girilen iki noktanın arasını bulur
    return int((p1.x + p2.x) / 2), int((p1.y + p2.y) / 2)

def hor(hor_points, facial_landmarks):
    # hor_points = [36, 39, 42, 45]

    left_left_point = (facial_landmarks.part(hor_points[0]).x, facial_landmarks.part(hor_points[0]).y)
    left_right_point = (facial_landmarks.part(hor_points[1]).x, facial_landmarks.part(hor_points[1]).y)

    right_left_point = (facial_landmarks.part(hor_points[2]).x, facial_landmarks.part(hor_points[2]).y)
    right_right_point = (facial_landmarks.part(hor_points[3]).x, facial_landmarks.part(hor_points[3]).y)

    left_hor_line_lenght = hypot((left_left_point[0] - left_right_point[0]),
                                  (left_left_point[1] - left_right_point[1]))
    right_hor_line_lenght = hypot((right_left_point[0] - right_right_point[0]),
                                   (right_left_point[1] - right_right_point[1]))

    hor_line_lenght = (left_hor_line_lenght + right_hor_line_lenght) / 2

    return hor_line_lenght

def get_blinking_ratio(eye_points, facial_landmarks):
    # sol göz = [36, 37, 38, 39, 40, 41] , sağ göz = [42, 43, 44, 45, 46, 47]
    center_top = midPoint(facial_landmarks.part(eye_points[1]), facial_landmarks.part(eye_points[2]))
    center_bottom = midPoint(facial_landmarks.part(eye_points[5]), facial_landmarks.part(eye_points[4]))

    hor_line_lenght = hor([36, 39, 42, 45], facial_landmarks)
    ver_line_lenght = hypot((center_top[0] - center_bottom[0]), (center_top[1] - center_bottom[1]))

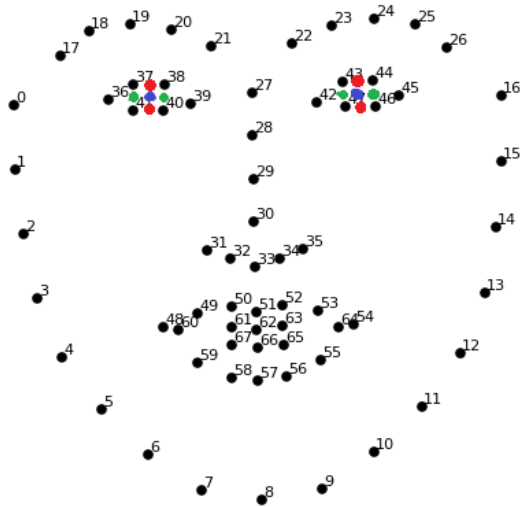
    ratio = round(hor_line_lenght / ver_line_lenght, 1)

    return ratio
```

## Kaş Kalkıklığı algılama

## Göz Merkezinin Hesaplanması

Göz üzerinde bulunan altışar noktanın dördü ortada kare şekilde oluşturacak şekilde bulunmaktadır (sol 37,38,40,41 sağ 43,44,46,47). Karşılıklı dikey kenarları oluşturan noktaların (yeşil) ortalaması bize gözün ortasının y koordinat değerini verirken karşılıklı yatay kenarlar (kırmızı) ise x koordinat değerini vermektedir böylece göz merkezi (mavi) hesaplanır.



```
def eyeCenter(eye_points, facial_landmarks):

    eyeXTop = ((facial_landmarks.part(eye_points[2]).x - facial_landmarks.part(
        eye_points[1]).x) / 2 + facial_landmarks.part(eye_points[1]).x)
    eyeXBottom = ((facial_landmarks.part(eye_points[4]).x - facial_landmarks.part(
        eye_points[5]).x) / 2 + facial_landmarks.part(eye_points[5]).x)

    eyeX = int((eyeXTop + eyeXBottom) // 2)

    eyeY1 = (int((facial_landmarks.part(eye_points[5]).y - facial_landmarks.part(
        eye_points[1]).y) // 2) + facial_landmarks.part(eye_points[1]).y)
    eyeY2 = (int((facial_landmarks.part(eye_points[4]).y - facial_landmarks.part(
        eye_points[2]).y) // 2) + facial_landmarks.part(eye_points[2]).y)

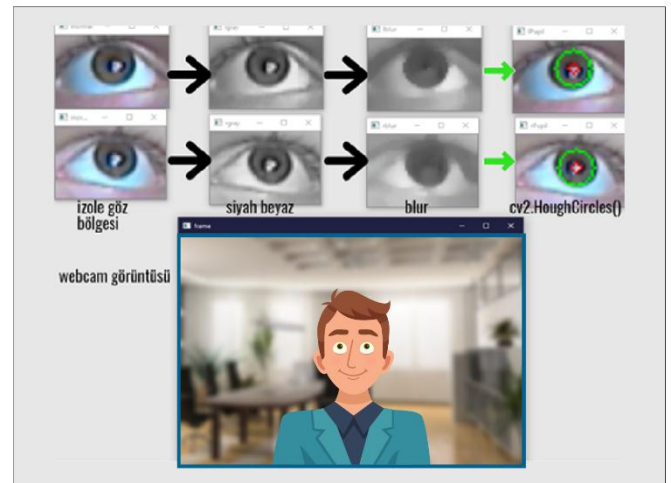
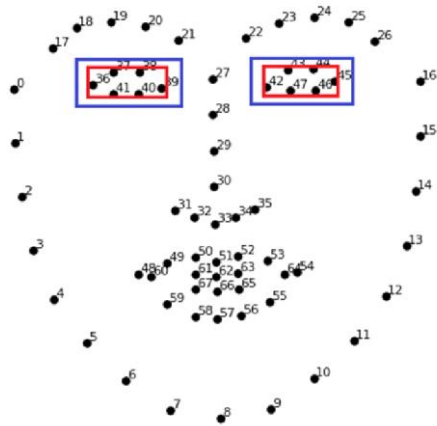
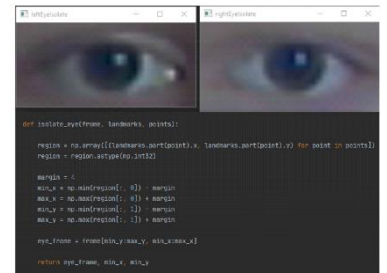
    eyeY = int((eyeY1 + eyeY2) // 2)

    eyeCoordinate = (eyeX, eyeY)

    return eyeCoordinate
```



Göz bebeği bulmak için göz üzerindeki noktalar kullanılarak bölge izole edilir (kırmızı) ve hata payı için genişletilen (mavi) görüntü üzerinde göz bebeği arama işlemleri yapılır. Görüntü siyah beyaza çevrilir sonra gürültüyü azaltmak için blurlanır. Son olarak görüntü üzerinde houghCircles metodu ile göz bebeği bulunur. (PDF SF 1)

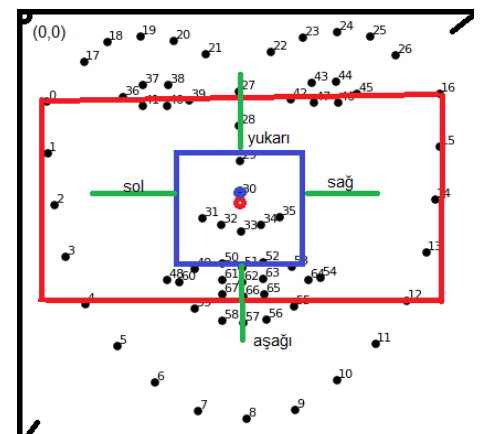
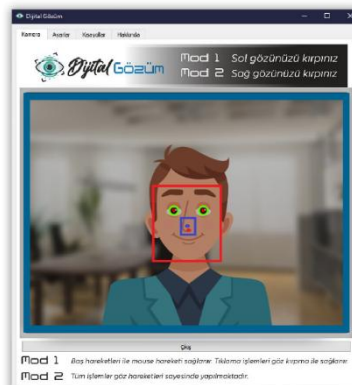
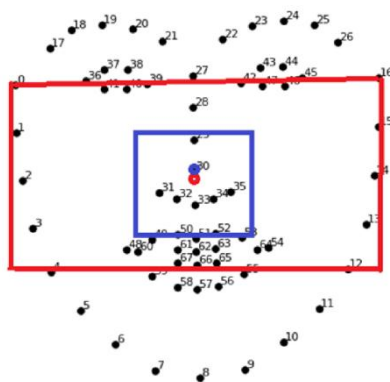


## Kısa yol işlemleri

İki gözün aynı anda kapalı olma zamanı kaydedilir ve son aynı anda kapanma zamanından çıkarılır eğer fark 5 saniye ve bu zaman içerisinde gözler kapalı ise kısa yol gerçekleştirilir ve kullanıcıya işitsel bildirimde bulunur. Diğer kısa yollar da aynı yöntem ile çalışmaktadır. (PDF SF2)

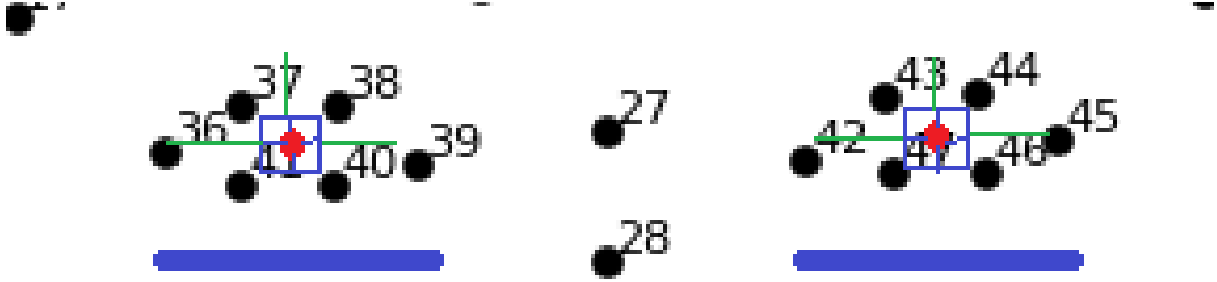
## Mod 1 Çalışma Prensipleri

Bu seçenekte baş hareketleri ile fare kontrolü sağlanmaktadır. Baş hareketleri tespiti için burunun ucu (30 mavi nokta) izlenmektedir. Yüz üzerindeki noktalar (0,16,12) arası işlemler yapılarak yüz kırmızı alan içine alınır bu alanın ortası noktası (kırmızı) yüzün ortasıdır. Yüzün orta noktasında işlemler yapılarak güvenli bölge belirlenir (mavi alan). Güvenli bölgeden kullanıcı kafasını hareket ettirmesi fare hareketine yansımaz. Burun bu bölgeden çıktığı yöne ve mavi bölgenin kenarına uzaklığı kat sayısı ile çarpılarak hareket ettirilir. Tüm kısayollar aktif şekilde kullanılabilir. (PDF SF3)



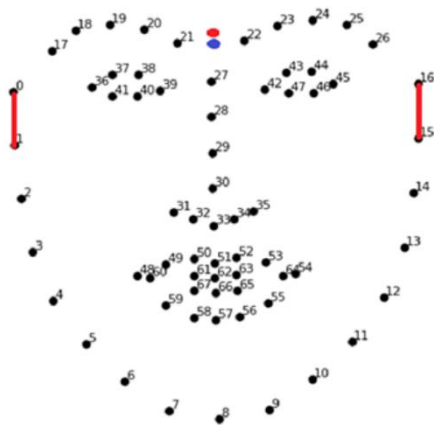
## Mod 2 Çalışma Prensipleri

Bulduğumuz göz merkezi üzerinde işlemler yaparak göz üzerinde güvenli bölge belirlenir (mavi alan). Güvenli bölgeden kullanıcının gözünü hareket ettirmesi fare hareketine yansımaz. İki gözün göz bebeğinin güvenli bölgenin kenarına olan uzaklığının ortalaması alınır katsayı ile çarpılır ve güvenli bölgeden çıktıkları yöne göre fare hareket ettirilir. Webcam in görüş açısından kaynaklı göz bebeğinin aşağı bakması gözükmeyeceği için kaşların kaldırılması ile fare aşağı hareket eder. (PDF SF4)



## Kaş hareketlerini algılama

Yüzün sol tarafının belli bir bölümünü uzunluğu (0,1) ile sağ tarafının belli bir uzunluğunun (15, 16) ortalaması alınır ve burnun kaşa en yakın noktası üzerinde işlemler yapılarak kaşın üstünde bir nokta oluşturulur (kırmızı). En yakın kaş noktalarının orta noktası bulunur (mavi) kaşlar kaldırıldığında kırmızı noktayı geçer ve bu duruma göre işlem yapılır.



```
def eyebrow(eye_points, landmarks):
    a = [21, 22, 27, 0, 1, 16, 15]

    mid = midpoint(landmarks.part(eye_points[0]), landmarks.part(eye_points[1]))
    cv2.circle(frame, mid, 1, blue, 1)
    margin = int(0.9 * ((landmarks.part(eye_points[5]).y - landmarks.part(eye_points[4]).y) +
                        (landmarks.part(eye_points[5]).y - landmarks.part(eye_points[0]).y)) // 2))
    cv2.circle(frame, (mid[0], landmarks.part(eye_points[2]).y + margin), 1, red, 1)

    if (landmarks.part(eye_points[2]).y + margin) > (mid[1]):
        eye_brow_up = 1
        return eye_brow_up
    else:
        eye_brow_up = 0
        return eye_brow_up
```

## Arayüz

Arayüz Pyqt5 ile yapılmıştır. 4 bölümden oluşmaktadır Kamera, Ayarlar, Kısayollar ve Hakkında. Kamera bölümünde kullanıcıya webcamden gelen görüntü gösterilmektedir. Ayarlar sekmesinde mod1 ve mod2 deki fare hareket hızını artırıp azaltılarak özelleştirilebilir. Bu programın kullanımına alışan kullanıcılar için önemli bir detaydır. Kullanıcılar alıştıktan sonra daha hızlı kullanmak isteyeceklerdir. Kısayollar bölümünde ise kısayolların nasıl çalışılacağı ve ne işe yaradıkları anlatılır. Hakkında bölümünde ise programın nasıl çalışacağı anlatılmaktadır. (PDF 5,6,7,8)

